

Health Care AI-Suite

Semester Project

Data Mining

DS-303



By

M. Faisal Kamran 2023-BSAI-025

Haris Awan 2023-BSAI-023

Hashir Afzaal 2023-BSAI-062

M. Hammad Ahmad 2023-BSAI-146

Submitted to

Mr. Muhammad Arham

Bachelor of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCE

Table of Contents

- 1. Executive Summary 3
- 2. Project Overview 3
 - 2.1 Objectives 3
 - 2.2 Scope 3
- 3. System Architecture..... 4
 - 3.1 High-Level Architecture 4
 - 3.2 Application Navigation 4
- 4. Module 1 — Patient Segmentation 4
 - 4.1 Purpose 4
 - 4.2 Data Preprocessing Pipeline 4
 - 4.3 Clustering Algorithms 5
 - 4.4 Evaluation Metrics 5
 - 4.5 Visualization 5
 - 4.6 Risk Scoring 5
 - 4.7 Export 5
- 5. Module 2 — Heart Attack Risk Predictor..... 6
 - 5.1 Purpose 6
 - 5.2 Data Preprocessing Pipeline 6
 - 5.3 Classification Algorithms 6
 - 5.4 Model Training Workflow 6
 - 5.5 Evaluation Metrics 7
 - 5.6 Visualizations 7
 - 5.7 Patient Search & Individual Risk Assessment 7
- 6. Technical Stack..... 8
 - 6.1 Core Libraries 8
 - 6.2 scikit-learn Components 8
- 7. Data Flow 8
- 8. User Interface & Experience Design 9
 - 8.1 Layout 9
 - 8.2 Custom CSS Styling 9
 - 8.3 Interactivity 9
- 9. Known Limitations & Future Work..... 9
 - 9.1 Current Limitations 9
 - 9.2 Recommended Future Enhancements 10
- 10. Conclusion 10

1. Executive Summary

The Healthcare AI Suite is a full-stack, interactive machine learning application built with Python and Streamlit. It combines two complementary clinical intelligence modules — Patient Segmentation and Heart Attack Risk Prediction — into a single, browser-based dashboard aimed at supporting data-driven decision-making in healthcare settings.

The system ingests real-world clinical datasets, applies rigorous preprocessing pipelines, and exposes configurable unsupervised and supervised machine learning workflows without requiring any programming knowledge from the end user. Clinicians and analysts can explore patient population structure, identify high-risk subgroups, train predictive models, and obtain per-patient risk assessments — all through an intuitive point-and-click interface.

2. Project Overview

2.1 Objectives

The Healthcare AI Suite was developed to achieve the following primary objectives:

- Automate patient stratification using unsupervised clustering to reveal hidden population patterns in stroke-risk data.
- Provide a configurable supervised classification pipeline for binary heart-attack risk prediction.
- Enable non-technical clinical staff to interact with machine learning models through a guided, no-code interface.
- Deliver interpretable outputs including risk scores, feature importances, and clinician-facing recommendations.
- Support data export for downstream use in EHR systems or further analysis.

2.2 Scope

The application operates on two distinct, publicly available healthcare datasets. The first is the Stroke Prediction Dataset (5,110 patient records, 11 clinical features), used for unsupervised patient segmentation. The second is the Heart Attack Prediction Dataset (8,763 patient records, 25 clinical features), used for supervised risk classification. Multi-user authentication and live EHR integration are outside the current scope.

3. System Architecture

3.1 High-Level Architecture

The application follows a modular monolithic architecture within a single Python file, structured around three logical layers: a data layer responsible for loading and preprocessing CSVs, a model layer that encapsulates all scikit-learn estimators and pipelines, and a presentation layer built entirely on Streamlit widgets, tabs, and reactive session state.

Layer	Responsibility	Key Technologies
Data Layer	CSV ingestion, null handling, feature engineering	pandas, SimpleImputer, LabelEncoder
Model Layer	Clustering, classification, evaluation metrics	scikit-learn, StandardScaler, PCA
Presentation Layer	Interactive UI, charts, session state management	Streamlit, Matplotlib, Seaborn

3.2 Application Navigation

A collapsible sidebar provides top-level navigation between the two functional modules. Navigation state is managed via Streamlit's radio widget bound to the `app_mode` variable, which conditionally renders one of two module functions.

4. Module 1 — Patient Segmentation

4.1 Purpose

The Patient Segmentation module applies unsupervised machine learning to the stroke dataset in order to group patients into clinically meaningful cohorts based on shared health characteristics. This supports population health management by revealing subgroups that may share elevated risk profiles even without explicit labels.

4.2 Data Preprocessing Pipeline

- Load `healthcare-dataset-stroke-data.csv` via `pandas`. File existence is validated before rendering.
- Drop the non-informative `id` column.
- Remove rows with missing values (`dropna`). Original and cleaned row counts are reported.
- Encode all categorical columns with `LabelEncoder` to produce integer representations.
- Scale all features to zero mean and unit variance using `StandardScaler`.
- If the `stroke` label column is present, it is separated from the feature matrix to keep clustering unsupervised.

4.3 Clustering Algorithms

Three clustering algorithms are available, each with configurable hyperparameters:

Algorithm	Hyperparameters	Best For
K-Means	n_clusters (2–8), n_init=10, random_state=42	Compact, spherical clusters of similar size
Hierarchical	n_clusters (2–8), linkage: ward / complete / average	Nested structure, unknown number of clusters
DBSCAN	eps (0.5–2.5), min_samples (2–10)	Arbitrary shapes, automatic noise detection

4.4 Evaluation Metrics

- Silhouette Score — measures intra-cluster cohesion vs inter-cluster separation. Range [-1, 1]; higher is better.
- Noise Point Count — for DBSCAN only: number and percentage of samples labelled as noise (label = -1).
- Cluster Count — number of distinct clusters found, always displayed regardless of algorithm.

4.5 Visualisation

When the feature dimensionality exceeds two, PCA reduces the scaled matrix to two components for plotting. The explained variance ratio is shown as a caption. A Matplotlib scatter plot colours points by cluster label using the tab10 colour map with a colour bar legend.

4.6 Risk Scoring

A heuristic risk score is computed per cluster by summing weighted binary conditions:

- Age > 60 years: +2 points
- BMI > 30 (obesity threshold): +2 points
- Mean hypertension rate x2: up to +2 points
- Mean heart disease rate x2: up to +2 points

Each cluster is classified as High Risk (score >= 4), Moderate Risk (score >= 2), or Low Risk (score < 2). A summary table of cluster size, risk score, and risk level is displayed interactively.

4.7 Export

A Download Results button enables users to export the original dataset augmented with a Cluster column as a CSV file (segmented_patients.csv) for use in external tools or reporting.

5. Module 2 — Heart Attack Risk Predictor

5.1 Purpose

The Heart Attack Risk Predictor implements a supervised binary classification workflow that trains a machine learning model on labelled patient data and estimates the probability that a given patient will experience a heart attack. The module also provides a patient search interface for on-demand individual risk assessment.

5.2 Data Preprocessing Pipeline

- Load heart_attack_prediction_dataset.csv. Patient ID columns are extracted separately for the search feature.
- Column names are normalised to lowercase with underscores.
- Blood pressure strings (format: 'systolic/diastolic') are parsed into two numeric columns.
- Categorical features are label-encoded; encoders are persisted in session state for inference.
- All columns are coerced to numeric; non-convertible values become NaN.
- Median imputation via SimpleImputer fills all remaining NaN values.
- StandardScaler normalises the imputed feature matrix.

5.3 Classification Algorithms

Algorithm	Key Configuration	Strength	Class Imbalance
Random Forest	100 trees, random_state=42	Accuracy + feature importance	class_weight='balanced'
Logistic Regression	max_iter=1000, random_state=42	Interpretable coefficients	class_weight='balanced'
Gradient Boosting	100 estimators, random_state=42	Sequential error correction	Manual handling

5.4 Model Training Workflow

- User configures algorithm and test split ratio (15%–35%) via sidebar controls.
- Stratified train-test split preserves class proportions in both subsets.
- Model training executes on button press with a spinner overlay during computation.
- Trained model, scaler, imputer, and encoders are stored in st.session_state for reuse.

5.5 Evaluation Metrics

Metric	Formula	Clinical Relevance
Accuracy	$(TP + TN) / \text{Total}$	Overall correctness
Precision	$TP / (TP + FP)$	False alarm rate control
Recall	$TP / (TP + FN)$	Missed positive case rate
F1 Score	$2 \times \text{Precision} \times \text{Recall} / (P + R)$	Balance of precision and recall
AUC-ROC	Area under the ROC curve	Overall discriminative power

5.6 Visualisations

- Confusion Matrix — Seaborn heatmap showing raw counts of true/false positives and negatives.
- Feature Importances — horizontal bar chart of top-10 features. Random Forest and Gradient Boosting use `model.feature_importances_`; Logistic Regression uses `model.coef_[0]`.

5.7 Patient Search & Individual Risk Assessment

After model training, a patient lookup interface allows search by Patient ID via text field or dropdown of the first 50 patients. The system retrieves the record, applies the full inference pipeline, and outputs a risk probability.

Risk Level	Probability	Recommendation
HIGH RISK	$\geq 70\%$	Immediate medical attention required. Consult a cardiologist urgently.
MODERATE RISK	40% – 69%	Schedule a check-up. Consider lifestyle modifications.
ELEVATED RISK	20% – 39%	Monitor regularly. Maintain healthy habits.
LOW RISK	$< 20\%$	Continue healthy lifestyle. Annual check-ups recommended.

Where available, the ground truth label is displayed alongside the prediction, enabling immediate accuracy verification on a per-patient basis.

6. Technical Stack

6.1 Core Libraries

Library	Version	Purpose
---------	---------	---------

Streamlit	Latest stable	Web UI framework, session state, interactive widgets
pandas	Latest stable	Data loading, manipulation, and transformation
NumPy	Latest stable	Numerical array operations
scikit-learn	Latest stable	Preprocessing, clustering, classification, evaluation
Matplotlib	Latest stable	Scatter plots, bar charts
Seaborn	Latest stable	Confusion matrix heatmap

6.2 scikit-learn Components Used

- Preprocessing: StandardScaler, LabelEncoder, SimpleImputer
- Dimensionality Reduction: PCA (2-component projection for visualisation)
- Clustering: KMeans, DBSCAN, AgglomerativeClustering
- Classification: RandomForestClassifier, LogisticRegression, GradientBoostingClassifier
- Model Selection: train_test_split (stratified), cross_val_score
- Metrics: accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score, silhouette_score, davies_bouldin_score

7. Data Flow

The end-to-end data flow for both modules follows a consistent five-stage pipeline:

#	Stage	Detail
1	Ingestion	CSV read via pandas.read_csv(). File existence validated before any UI renders.
2	Preprocessing	Null removal or imputation, categorical encoding, blood pressure parsing, feature/target separation.
3	Scaling	StandardScaler transforms all numeric features to zero mean and unit variance.
4	Modelling	User-selected algorithm is instantiated with chosen hyperparameters and fitted to training data.
5	Output	Results, metrics, and charts rendered in Streamlit. Inference run on selected patient records.

8. User Interface & Experience Design

8.1 Layout

The application uses Streamlit's wide layout mode to maximise screen real estate. Content is organised into multi-column grids using `st.columns()`, ensuring metric cards, controls, and charts appear side by side.

8.2 Custom CSS Styling

A custom CSS block injected via `st.markdown(unsafe_allow_html=True)` applies a professional aesthetic:

- Gradient buttons (purple-to-violet) with hover lift animation via CSS transform and box-shadow transition.
- Colour-coded risk cards (red for HIGH, amber for MODERATE, green-teal for LOW).
- Rounded corners (border-radius: 10–12px) on cards, dataframes, and alerts.
- Sidebar with dark navy (#2c3e50) background to visually separate navigation from content.
- Tabs styled with padding, rounded corners, and a light grey background.

8.3 Interactivity

- Model configuration controls are rendered in an expandable expander that collapses automatically after training.
- `st.session_state` persists trained models, scalers, encoders, and imputers across user interactions.
- Spinner overlays (`st.spinner`) provide visual feedback during computation-intensive operations.
- Warning and error banners guide users when required dataset files are missing.

9. Known Limitations & Future Work

9.1 Current Limitations

- The `os` module import is placed mid-file (line 183) rather than at the top — a PEP 8 violation that could cause `NameError` in some execution paths.
- `LabelEncoder` is fitted on the full dataset before any train-test split, introducing minor data leakage for supervised tasks.
- Patient segmentation risk scoring uses fixed, manually defined thresholds rather than data-driven boundaries.
- No persistent storage: all trained models are held in `st.session_state` and are lost on browser refresh or server restart.
- Single-user design: no authentication, role-based access control, or audit logging.
- DBSCAN noise points (`label = -1`) are excluded from the silhouette score computation, which may understate quality.

9.2 Recommended Future Enhancements

- Introduce a formal `sklearn.pipeline.Pipeline` to eliminate data leakage and simplify maintenance.

- Persist trained models to disk via `joblib.dump` so sessions can be restored without retraining.
- Add cross-validation metrics (k-fold CV scores, learning curves) alongside hold-out evaluation.
- Integrate SHAP values for clinically interpretable, patient-level feature attribution.
- Build a REST API layer (FastAPI or Flask) to decouple the ML backend from the Streamlit frontend.
- Add automated hyperparameter tuning (GridSearchCV or Optuna) accessible via a UI toggle.

10. Conclusion

The Healthcare AI Suite successfully delivers an end-to-end clinical analytics platform that bridges the gap between raw health data and actionable medical insight. By combining unsupervised patient segmentation with supervised heart attack risk prediction in a single, accessible web application, the system demonstrates how modern Python ML tooling can be packaged for non-technical healthcare professionals.

The modular codebase, comprehensive preprocessing pipelines, multi-algorithm support, and rich interactive visualisations make this a solid foundation for a production clinical decision support tool. With the enhancements outlined in Section 9, the system could be extended into a robust, compliant, and fully integrated healthcare AI platform.